

UNIVERSIDAD NACIONAL DE JULIACA
Facultad de Ciencias de Ingenierías
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE Y SISTEMAS



MONOGRAFIA

TITULO
VERIFICACION FORMAL BASICA DE ALGORITMOS USANDO LOGICA
PROPOSICIONAL

ASIGNATURA
FUNDAMENTOS Y METODOLOGÍA DE LA PROGRAMACIÓN

CICLO

III

ESTUDIANTE
DIANA MILAGROS TINTAYA MAMANI

ASESOR
JUAN CARLOS PINTO LARICO

JULIACA – PERÚ

2026-I

RESUMEN

La presente monografía desarrolla una verificación formal básica de algoritmos usando lógica, aplicada a un mini sistema bancario implementado en C++. El objetivo principal es demostrar cómo las reglas condicionales de un programa pueden representarse mediante proposiciones, conectivos lógicos, tablas de verdad y precondiciones/postcondiciones. La metodología empleada es teórico-experimental y de desarrollo, debido a que se revisan conceptos del curso de Estructuras Discretas I, como lógica proposicional, tablas de verdad, equivalencias lógicas, reglas de inferencia, conjuntos y relaciones; además, se implementa un caso de estudio funcional. El mini sistema bancario considera operaciones de inicio de sesión, depósito, retiro y transferencia. Cada operación se modela mediante fórmulas lógicas, por ejemplo, la regla de retiro se expresa como $(A \wedge M \wedge F) \rightarrow R$, donde A representa cuenta activa, M monto positivo, F fondos suficientes y R retiro realizado. Los resultados muestran que la verificación mediante tabla de verdad permite evaluar todas las combinaciones posibles de entrada y obtener una cobertura lógica del 100 % en la regla analizada. Se concluye que la lógica proposicional es una herramienta útil para detectar errores en algoritmos condicionales y mejorar la confiabilidad del software.

Palabras clave: lógica proposicional, verificación formal, algoritmos, C++, sistema bancario, tablas de verdad.

INDICE

RESUMEN.....	2
INDICE	3
INTRODUCCIÓN	5
1.1 DESCRIPCION DE LA PROBLEMÁTICA:	6
Problema principal	6
1.2 JUSTIFICACIÓN E IMPORTANCIA	6
1.3 OBJETIVOS	6
Objetivo general	6
Objetivo específico.....	6
MARCO TEORICO.....	7
2.1 BASES TEORICAS O CIENTIFICAS.....	7
Lógica matemática	7
Conjuntos, relaciones y casos de prueba	8
2.2. DEFINICION DE TERMINOS BÁSICOS	8
Algoritmo	8
Tautología	8
Contradicción	9
Modus Ponens	9
Modus Tollens.....	9
Precondición.....	9
Postcondición	9
Terna de Hoare.....	10
Cobertura lógica	10
METODOLOGIA	10
3.1. ENFOQUE	10
3.2. DISEÑO	11
3.3. MUESTRA.....	11
Muestra.....	11
Caso de estudio	12
Técnicas.....	12
DESARROLLO E IMPLEMENTACION	13
4.1. MODELO GENERAL DEL SISTEMA	13
Proposiciones simples	13
4.2 REPRESENTACIÓN DEL ALGORITMO MEDIANTE FÓRMULAS PROPOSICIONALES.....	13

4.3 DEMOSTRACIÓN CON EQUIVALENCIAS LÓGICAS	14
4.4 GENERACIÓN DE TODOS LOS CASOS POSIBLES USANDO TABLAS DE VERDAD	16
4.3 PSEUDOCÓDIGO DE RETIRO	16
4.4 DIAGRAMA LÓGICO.....	17
4.5 TERNA DE HOARE APLICADA AL RETIRO.....	18
RESULTADOS.....	18
5.1. CASOS GENERADOS.....	18
5.2. TABLA DE VERDAD DE LA REGLA DE RETIRO	19
5.3. MÉTRICAS DE VERIFICACIÓN	20
DISCUSION	21
CONCLUSION	22
REFERENCIAS	23
ANEXOS.....	24
Anexo A. Código fuente.....	24
Anexo B. Pruebas realizadas	24
Anexo E. Evidencia del repositorio.....	25

INTRODUCCIÓN

En un mundo cada vez más digitalizado, los entornos virtuales dejan de verse como un lujo de los usuarios y se convierten en una necesidad del día a día para cada trabajo u ocupación que tenga la persona. El desarrollo de estos entornos de software requiere la aplicación de una estructura lógica, resultando en las aplicaciones, programas o páginas web que usan miles de usuarios; por ello, el pilar es la lógica de programación empleada. Antes de la escritura de un código en Python o C++, los desarrolladores de software diseñan los algoritmos para asegurar que el software sea eficiente. La validación de un software robusto se confirma con la ejecución de estas estructuras lógicas, las cuales encuentran su sustento en la lógica proposicional.

Esta disciplina constituye una base formal para analizar condiciones, reglas de decisión y estructuras de control en programación. Precisamente, en el curso de Estructuras Discretas I; la lógica matemática se presenta como una base para la programación, las bases de datos, la verificación de software, la inteligencia artificial y el diseño de algoritmos. En las diapositivas del curso se expone que la lógica proposicional se trabaja como el fundamento del razonamiento formal, la construcción de tablas de verdad, la simplificación de expresiones lógicas y la verificación de programas. Además, desarrollan tautologías, contradicciones, contingencias y leyes de equivalencia; conceptos matemáticos necesarios para determinar si una regla lógica es válida, inconsistente o dependiente de los valores de entrada, garantizando así la robustez absoluta del software antes de su puesta en marcha.

El presente trabajo se enfoca en la verificación formal básica de un mini sistema bancario implementado en C++. Se eligió este caso porque las operaciones bancarias requieren reglas claras: una cuenta debe estar activa para operar, un retiro solo debe permitirse si existe saldo suficiente y una transferencia debe modificar correctamente los

saldos de origen y destino; ello puede ser expresado mediante fórmulas de lógica proposicional y evaluarse con tablas de verdad.

1.1 DESCRIPCION DE LA PROBLEMÁTICA:

En la construcción de software, los programadores muchas veces realizan el desarrollo de algoritmos básicos sin ninguna verificación; se aplican de forma conceptual y directa en el código sin una validación estructural previa. Ello provoca fallas sutiles e imperceptibles para la prueba o *testing*.

Problema principal

¿De qué manera la aplicación de la lógica proposicional permite realizar la verificación formal de un algoritmo implementado en un mini sistema bancario en C++?

1.2 JUSTIFICACIÓN E IMPORTANCIA

El trabajo se justifica por su relevancia y aplicación en la carrera de Software porque explica como el cimiento de la programación fue aplicado y como sigue siendo importante la lógica proposicional en la verificación y evaluación de condiciones críticas. Además, la formación del estudiante es optimizado por el razonamiento de algoritmos y permite una mayor critica a la detección de errores lógicos frente al diseño de software.

1.3 OBJETIVOS

Objetivo general

Aplicar lógica proposicional para verificar formalmente un algoritmo básico de control de acceso.

Objetivo específico

- Identificar proposiciones simples dentro de una regla algorítmica.
- Representar el algoritmo mediante fórmulas proposicionales.

- Generar todos los casos posibles usando tablas de verdad.
- Implementar un verificador básico en Python.
- Evaluar cobertura lógica, cumplimiento de especificación y limitaciones.

MARCO TEORICO

2.1 BASES TEORICAS O CIENTIFICAS

Lógica matemática

La lógica matemática estudia la estructura de los razonamientos válidos. En computación, permite representar decisiones, condiciones, reglas de negocio y procesos de verificación. En este caso en el sistema bancario; existe condiciones que pueden ser presentadas como una expresión lógica, tal cual seria “si la cuenta está activa y hay fondos suficientes, entonces se permite el retiro”, siendo observables y de análisis antes de ser ejecutas en código.

Lógica proposicional

La lógica proposicional trabaja con proposiciones que pueden tomar solo dos valores: verdadero o falso. Una proposición puede ser simple o compuesta dependiendo de los conectores.

Tautologías, contradicciones y contingencias

Una tautología es una proposición compuesta verdadera en todos los casos. Una contradicción es falsa en todos los casos. Una contingencia puede ser verdadera o falsa dependiendo de los valores de sus variables. El curso desarrolla estos conceptos como parte del análisis de tablas de verdad.

Verificación formal de algoritmos

El libro *Logic in Computer Science* de Huth y Ryan se enfoca en modelar y razonar sobre sistemas computacionales usando lógica formal. Por su parte, *Principles of Model Checking* presenta el model checking como una técnica automatizada para encontrar fallos en hardware y software.

Conjuntos, relaciones y casos de prueba

La teoría de conjuntos permite modelar dominios de entrada y posibles estados del sistema. En el curso se define un conjunto como una colección de elementos distintos, y se trabaja la diferencia entre pertenencia e inclusión.

En este informe, el conjunto de valores booleanos es:

$$B = \{V, F\}$$

Para la regla de retiro, el conjunto de casos posibles se obtiene mediante el producto cartesiano:

$$B \times B \times B$$

Esto genera las ocho combinaciones posibles de A , M y F .

Por otro lado, el curso define una relación binaria R como un subconjunto del producto cartesiano $A \times B$. En el mini sistema bancario, cada operación puede interpretarse como una relación entre el conjunto de entradas booleanas y el conjunto de salidas posibles.

2.2. DEFINICION DE TERMINOS BÁSICOS

Algoritmo

Conjunto ordenado y finito de pasos que permite resolver un problema. En este informe, el algoritmo principal corresponde a las operaciones del mini sistema bancario: inicio de sesión, depósito, retiro y transferencia.

Tautología

Proposición compuesta que resulta verdadera en todos los casos posibles.

Contradicción

Proposición compuesta que resulta falsa en todos los casos posibles.

Modus Ponens

Regla de inferencia con la forma:

$$p \rightarrow q$$

$$p$$

$$\therefore q$$

En el sistema bancario, si se cumple la condición del retiro, entonces el retiro se realiza.

Modus Tollens

Regla de inferencia con la forma:

$$p \rightarrow q$$

$$\neg q$$

$$\therefore \neg p$$

En el sistema bancario, si el retiro no se realiza, se deduce que alguna condición necesaria no se cumplió.

Precondición

Condición que debe cumplirse antes de ejecutar una operación. Para retirar dinero, una precondición es que el saldo sea suficiente.

Postcondición

Condición que debe cumplirse después de ejecutar una operación. Para retirar dinero, una postcondición es que el saldo final sea igual al saldo inicial menos el monto retirado.

Terna de Hoare

Representación formal de una operación mediante:

$$\{P\} C \{Q\}$$

donde P es la precondition, C es el código y Q es la postcondición.

Sistema bancario

Cobertura lógica

Porcentaje de casos evaluados respecto al total de casos posibles. Se calcula así:

$$Cobertura = \frac{\text{casos evaluados}}{\text{casos posibles}} \times 100$$

En la regla de retiro se evaluaron ocho de ocho casos posibles, por lo que la cobertura fue del 100 %.

METODOLOGIA

3.1. ENFOQUE

El enfoque del trabajo es teórico-experimental y de desarrollo. Es teórica porque revisa conceptos de lógica proposicional, tablas de verdad, leyes de equivalencia, reglas de inferencia, conjuntos, relaciones y verificación formal. Es experimental porque implementa un mini sistema bancario en C++ y evalúa el comportamiento lógico de sus operaciones. Es también un trabajo de **desarrollo**, porque se construye una solución computacional mediante Programación Orientada a Objetos. Para ello, se diseñan clases que representan las cuentas bancarias, el sistema bancario y el verificador lógico. De esta manera, el trabajo no se limita a la teoría, sino que aplica los conceptos lógicos en un caso práctico de software.

3.2. DISEÑO

El diseño del trabajo corresponde a un **diseño aplicado, experimental básico y descriptivo**. El diseño se organiza en 4 fases:

Fase	Descripción
Fase 1: Análisis lógico	Identificación de proposiciones.
Fase 2: Formalización	Representación de fórmulas proposicionales
Fase 3: Implementación	Desarrollo del mini sistema bancario en C++ usando Programación Orientada a Objetos.
Fase 4: Verificación	Evaluación mediante tablas de verdad y postcondiciones

HERRAMIENTAS UTILIZADAS

Herramienta	Uso dentro del trabajo
C++	Implementación del mini sistema bancario
Compilador g++	Compilación y ejecución del programa
GitHub	Repositorio del código fuente y evidencias

3.3. MUESTRA

Muestra

La muestra está conformada por los **casos lógicos generados a partir de las combinaciones booleanas** de la regla de retiro del mini sistema bancario.

La regla principal evaluada es:

$$(A \wedge M \wedge F) \rightarrow R$$

Como existen tres variables proposicionales de entrada, el número de combinaciones se calcula con:

$$2^n$$

$$2^3 = 8$$

Por tanto, la muestra está formada por **8 casos de prueba lógicos**.

Caso de estudio

El caso de estudio corresponde a un **mini sistema bancario desarrollado en C++ bajo Programación Orientada a Objetos**.

El sistema contiene dos cuentas bancarias y permite realizar las siguientes operaciones: Inicio de sesión, Depósito, Retiro y Transferencia.

La operación principal verificada es el **retiro**, porque modifica el saldo de la cuenta y requiere condiciones críticas. Si el retiro se ejecuta sin validar correctamente la cuenta activa, el monto positivo o los fondos suficientes, el sistema podría producir un saldo incorrecto o permitir una operación inválida.

Técnicas

Técnica	Aplicación
Revisión documental	Revisión de diapositivas del curso, libros, artículos y fuentes académicas
Modelado lógico	Conversión de reglas bancarias a proposiciones
Evaluación por tabla de verdad	Generación de los 8 casos posibles
Verificación por postcondición	Comprobación del saldo final luego del retiro
Prueba de ejecución	Ejecución del programa en C++ y registro de resultados

DESARROLLO E IMPLEMENTACION

4.1. MODELO GENERAL DEL SISTEMA

El algoritmo de retiro solo debe permitir la operación si se cumplen tres condiciones:

1. La cuenta está activa.
2. El monto ingresado es positivo.
3. Existen fondos suficientes.

Si una de estas condiciones no se cumple, el retiro debe ser rechazado.

Proposiciones simples

SÍMBOLO	PROPOSICIÓN SIMPLE	VALOR POSIBLE
<i>A</i>	La cuenta está activa	<i>V/F</i>
<i>M</i>	El monto ingresado es positivo	<i>V/F</i>
<i>F</i>	Existen fondos suficientes	<i>V/F</i>
<i>R</i>	El retiro se realiza	<i>V/F</i>

En esta etapa se identifica a las proposiciones simples de la condición del algoritmo. Para el algoritmo las 3 primeras proposiciones vendrían a ser condiciones de entrada y la ultima proposición el resultado esperado de la operación

4.2 REPRESENTACIÓN DEL ALGORITMO MEDIANTE FÓRMULAS PROPOSICIONALES

Ahora pasas del algoritmo a la lógica.

El pseudocódigo sería:

```
Si A y M y F entonces
    realizar retiro
Si no
    rechazar retiro
Fin si
```

La condición compuesta es:

$$A \wedge M \wedge F$$

La regla de verificación básica queda:

$$(A \wedge M \wedge F) \rightarrow R$$

La regla algorítmica se representa mediante una **implicación lógica**. El antecedente está formado por la conjunción de las condiciones necesarias para realizar el retiro: cuenta activa, monto positivo y fondos suficientes. El consecuente representa el resultado esperado: que el retiro se realice. Por tanto, la regla formal del algoritmo es:

$$(A \wedge M \wedge F) \rightarrow R$$

Pero aquí puedes mejorar la demostración agregando una **especificación más completa**:

$$R \leftrightarrow (A \wedge M \wedge F)$$

Esta fórmula es mejor para verificación porque dice:

El retiro se realiza **si y solo si** se cumplen las tres condiciones.

Puedes explicar así:

<i>Fórmula</i>	<i>Qué verifica</i>
$(A \wedge M \wedge F) \rightarrow R$	Si las condiciones se cumplen, el retiro debe realizarse.
$R \rightarrow (A \wedge M \wedge F)$	Si el retiro se realizó, entonces las condiciones debieron cumplirse.
$R \leftrightarrow (A \wedge M \wedge F)$	El retiro ocurre únicamente cuando las condiciones son verdaderas.

4.3 DEMOSTRACIÓN CON EQUIVALENCIAS LÓGICAS

Aquí es donde se verá más el desarrollo matemático del curso.

Partimos de la regla:

$$(A \wedge M \wedge F) \rightarrow R$$

Usamos la equivalencia de la implicación:

$$p \rightarrow q \equiv \neg p \vee q$$

Entonces:

$$(A \wedge M \wedge F) \rightarrow R$$

Aplicando la ley de De Morgan:

$$\equiv (\neg A \vee \neg M \vee \neg F) \vee R$$

Por asociatividad:

$$\equiv \neg A \vee \neg M \vee \neg F \vee R$$

Mediante equivalencias lógicas, la regla de retiro puede transformarse sin alterar su valor de verdad. La implicación $(A \wedge M \wedge F) \rightarrow R$ equivale a $\neg(A \wedge M \wedge F) \vee R$. Luego, aplicando la ley de De Morgan, se obtiene $\neg A \vee \neg M \vee \neg F \vee R$. Esta forma permite observar que la regla solo sería falsa si A , M y F fueran verdaderas, pero R fuera falsa.

La condición de error lógico es:

$$A \wedge M \wedge F \wedge \neg R$$

Interpretación:

Existe error si la cuenta está activa, el monto es positivo y hay fondos suficientes, pero el retiro no se realiza.

4.4 GENERACIÓN DE TODOS LOS CASOS POSIBLES USANDO TABLAS DE VERDAD

Como hay tres variables de entrada: A , M , F . y cada una puede tomar dos valores:

$$B = \{V, F\}$$

entonces el conjunto de casos posibles es:

$$B^3 = B \times B \times B$$

El número total de casos es: $2^n = 2^3 = 8$

<i>Caso</i>	<i>A</i>	<i>M</i>	<i>F</i>	$A \wedge M \wedge F$	<i>R</i> esperado	Fórmula $(A \wedge M \wedge F) \rightarrow R$	¿Cumple?
1	F	F	F	F	F	V	Sí
2	F	F	V	F	F	V	Sí
3	F	V	F	F	F	V	Sí
4	F	V	V	F	F	V	Sí
5	V	F	F	F	F	V	Sí
6	V	F	V	F	F	V	Sí
7	V	V	F	F	F	V	Sí
8	V	V	V	V	V	V	Sí

4.3 PSEUDOCÓDIGO DE RETIRO

Algoritmo Retirar

Entrada: cuenta, monto

Definir A como cuenta activa

Definir M como monto positivo

Definir F como fondos suficientes

Definir R como retiro realizado

Si A Y M Y F Entonces

 saldo = saldo - monto

 R = Verdadero

Si no


```

    R = Falso
Fin Si

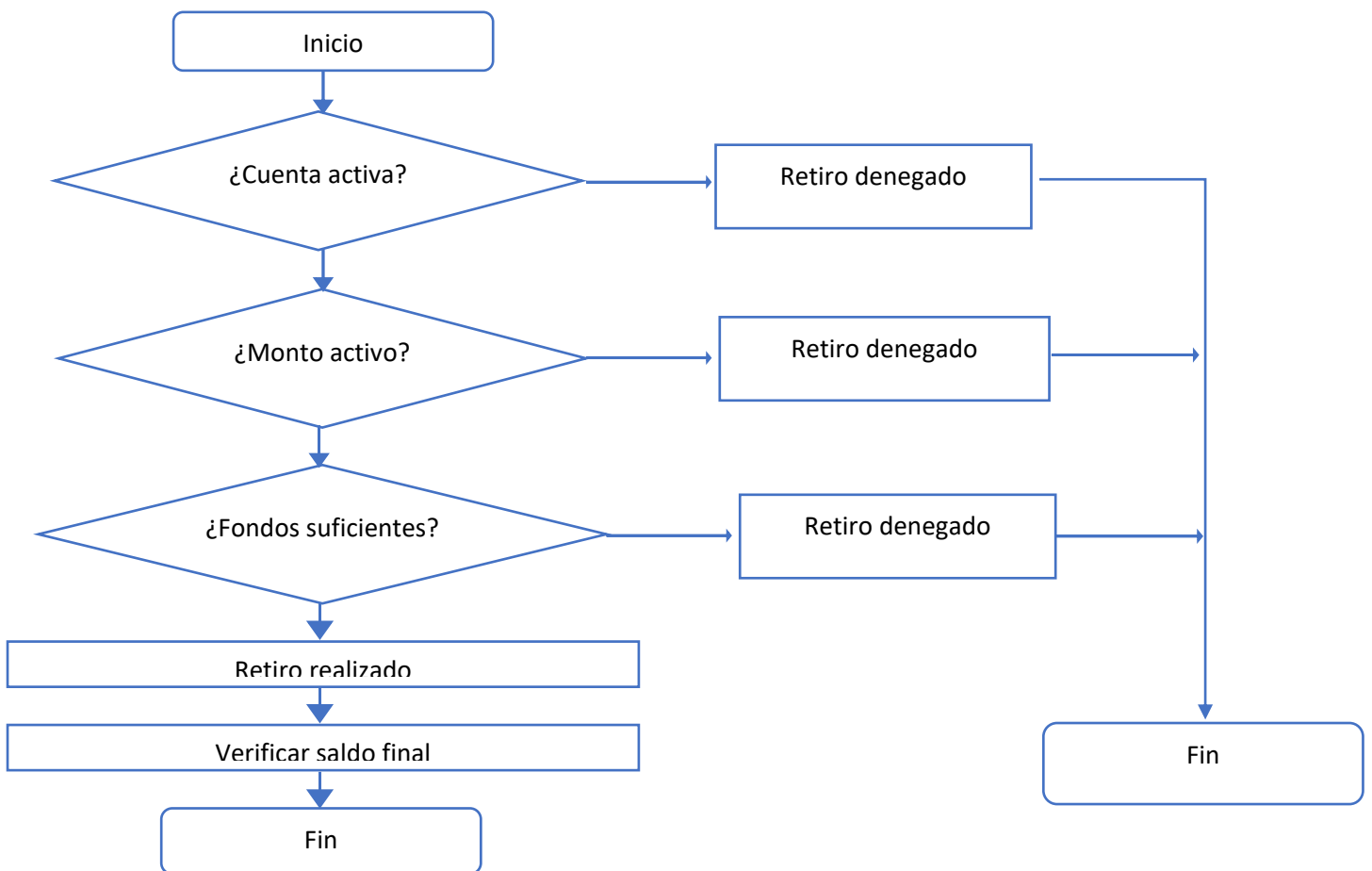
Verificar:
(A ∧ M ∧ F) → R

Si R es Verdadero Entonces
    Verificar postcondición:
    saldoFinal = saldoInicial - monto
    saldoFinal >= 0
Fin Si

Fin Algoritmo

```

4.4 DIAGRAMA LÓGICO



4.5 TERNA DE HOARE APLICADA AL RETIRO

La operación de retiro puede representarse mediante una terna de Hoare:

$$\{cuentaActiva \wedge monto > 0 \wedge saldoInicial \geq monto\}$$

```
saldo = saldo - monto;
```

$$\{saldoFinal = saldoInicial - monto \wedge saldoFinal \geq 0\}$$

La precondition establece que, antes de ejecutar el retiro, la cuenta debe estar activa, el monto debe ser positivo y el saldo inicial debe ser mayor o igual al monto solicitado. La instrucción modifica el saldo de la cuenta. La postcondition exige que el saldo final sea igual al saldo inicial menos el monto retirado y que no quede en negativo.

Esta parte fortalece el trabajo porque no solo verifica una condición lógica, sino también el efecto producido por la operación sobre el estado del sistema. En verificación formal, esta relación entre estado inicial, código y estado final es fundamental para razonar sobre la corrección de programas.

RESULTADOS

5.1. CASOS GENERADOS

En esta sección se presentan los resultados obtenidos a partir del modelo lógico aplicado al mini sistema bancario desarrollado en C++. La operación principal evaluada fue el **retiro**, porque modifica el saldo de una cuenta y requiere validar condiciones críticas antes de ejecutarse.

La regla lógica evaluada fue:

$$(A \wedge M \wedge F) \rightarrow R)$$

Donde:

- A La cuenta está activa.
- M El monto ingresado es positivo.
- F Existen fondos suficientes.
- R El retiro se realiza.

Donde n es el número de variables proposicionales.

$$2^3 = 8$$

Por tanto, se generan 8 casos lógicos. Este cálculo se justifica porque las tablas de verdad evalúan todas las combinaciones posibles de una proposición compuesta; en el material del curso se indica que con n variables se generan 2^n combinaciones.

5.2. TABLA DE VERDAD DE LA REGLA DE RETIRO

La tabla de verdad permite evaluar todos los escenarios posibles de la regla de retiro. En el modelo correcto, el retiro RRR solo debe realizarse cuando la cuenta está activa, el monto es positivo y existen fondos suficientes.

Caso	A:	M:	F:	$(A \wedge M \wedge F)$	R:	$(A \wedge M \wedge F) \rightarrow R$
1	F	F	F	F	F	V
2	F	F	V	F	F	V
3	F	V	F	F	F	V
4	F	V	V	F	F	V
5	V	F	F	F	F	V
6	V	F	V	F	F	V
7	V	V	F	F	F	V
8	V	V	V	V	V	V

La fórmula resultó verdadera en los 8 casos. Esto significa que la regla de retiro cumple la especificación lógica planteada: el retiro se realiza únicamente cuando las tres condiciones necesarias son verdaderas.

5.3. MÉTRICAS DE VERIFICACIÓN

Las métricas usadas fueron: número de casos posibles, número de casos evaluados, casos correctos, casos fallidos y cobertura lógica.

La cobertura lógica se calcula así:

$$Cobertura = \frac{\text{casos evaluados}}{\text{casos posibles}} \times 100$$

Reemplazando:

$$Cobertura = \frac{8}{8} \times 100$$

$$Cobertura = 100$$

Además de la fórmula proposicional, se considera la postcondición de retiro:

$$\text{saldoFinal} = \text{saldoIncial} - \text{monto}$$

y el invariante de seguridad:

$$\text{saldoFinal} \geq 0$$

Esto permite verificar que el retiro no solo cumpla una condición lógica previa, sino que también produzca un estado final válido. En tus diapositivas de demostración formal se presenta la forma $\{P\} C \{Q\}$ donde una precondición verdadera antes de ejecutar el código debe conducir a una postcondición verdadera después de la ejecución.

DISCUSION

Los resultados muestran que la lógica proposicional permite verificar reglas condicionales críticas dentro de un mini sistema bancario. En la operación de retiro, la condición $A \wedge M \wedge F$ funciona como una precondition de seguridad: la cuenta debe estar activa, el monto debe ser positivo y los fondos deben ser suficientes. Si una de estas condiciones no se cumple, el retiro debe ser denegado.

La tabla de verdad permitió evaluar los 8 casos posibles. En todos ellos, la fórmula $(A \wedge M \wedge F) \rightarrow R$ resultó verdadera. Esto indica que el algoritmo cumple la regla lógica definida para el retiro. Además, se incorporó una postcondición para comprobar que, cuando el retiro se realiza, el saldo final sea igual al saldo inicial menos el monto retirado y no quede en negativo.

En el mismo sentido, el curso de Estructuras Discretas I presenta la lógica de Hoare con la forma $\{P\} C \{Q\}$, aplicada a la verificación de programas.

Una limitación importante del método aplicado es el crecimiento exponencial de casos. Para tres variables se generan ocho combinaciones; sin embargo, si se usaran diez variables, se generarían:

$$2^{10} = 1024$$

casos. Esto demuestra que las tablas de verdad son útiles para reglas pequeñas, pero pueden volverse poco prácticas en sistemas con muchas condiciones. En verificación formal avanzada, técnicas como el model checking permiten analizar automáticamente si un sistema cumple propiedades especificadas; Baier y Katoen describen el model checking como una técnica automatizada para encontrar fallos en hardware y software.

CONCLUSION

La lógica proposicional permitió representar formalmente la operación de retiro de un mini sistema bancario mediante la fórmula $(A \wedge M \wedge F) \rightarrow R$.

La tabla de verdad generó 8 casos posibles, correspondientes a las combinaciones de las tres variables proposicionales A, M y F.

La fórmula evaluada resultó verdadera en todos los casos, por lo que la regla de retiro cumplió la especificación lógica definida.

La cobertura lógica fue del 100 %, ya que se evaluaron los 8 casos posibles.

El uso de postcondiciones permitió verificar que, cuando el retiro se realiza, el saldo final debe ser igual al saldo inicial menos el monto retirado y no debe quedar en negativo.

La Programación Orientada a Objetos permitió organizar el mini sistema bancario en clases, separando las responsabilidades de cuenta bancaria, sistema bancario y verificación lógica.

El trabajo demuestra que los contenidos de Estructuras Discretas pueden aplicarse directamente en el diseño y validación de software. El principal aporte del trabajo es mostrar una forma básica y comprensible de aplicar verificación formal en un sistema de software. Aunque el sistema bancario implementado es pequeño, permite observar cómo una regla de negocio puede expresarse como fórmula lógica, evaluarse mediante tabla de verdad y comprobarse mediante postcondiciones.

Este análisis contribuye a mejora en la calidad del software y una mejor validación de software.

Aporte Descripción

Mejora de calidad del software Permite detectar errores lógicos antes de escalar el sistema.

REFERENCIAS

- Álvarez, J. V. (s. f.). *Verificación formal de algoritmos*. Universidad de Valladolid.
<https://www.infor.uva.es/~jvalvarez/docencia/tema8.pdf>
- Baier, C., & Katoen, J.-P. (2008). *Principles of model checking*. MIT Press.
<https://mitpress.mit.edu/9780262026499/principles-of-model-checking/>
- Hoare, C. A. R. (1969). An axiomatic basis for computer programming.
Communications of the ACM, 12(10), 576–580. <https://doi.org/10.1145/363235.363259>
- Huth, M., & Ryan, M. (2004). *Logic in computer science: Modelling and reasoning about systems* (2nd ed.). Cambridge University Press.
- Pinto L., J. C. (2026). *Fundamentos de la lógica matemática: Enunciados y conectivos*
[Diapositivas de clase]. Estructuras Discretas I, Ingeniería de Software y Sistemas.
- Pinto L., J. C. (2026). *Tautologías, contradicciones y leyes de equivalencia*
[Diapositivas de clase]. Estructuras Discretas I, Ingeniería de Software y Sistemas.
- Pinto L., J. C. (2026). *Métodos de demostración formal y reglas de inferencia*
[Diapositivas de clase]. Estructuras Discretas I, Ingeniería de Software y Sistemas.

ANEXOS

Anexo A. Código fuente

https://github.com/milagrostintayam-crypto/ESTRUCTURAS_DISCRETAS

Anexo B. Pruebas realizadas

1. Compilación del programa.
2. Ejecución del programa.
3. Resultado de la tabla de verdad.

```
● PS D:\ESTRUCTURAS_DISCRETAS> .\main.exe
MINI BANCO - VERIFICACION FORMAL BASICA
Juan Perez | Saldo: S/ 1000 | Activa: V
Maria Lopez | Saldo: S/ 500 | Activa: V

[LOGIN]
(P and A) entonces L = V

[DEPOSITO]
(A and M) entonces D = V

[RETIRO]
A: V | M: V | F: V | R: V
(A and M and F) entonces R = V
Postcondicion saldo valido = V

[TRANSFERENCIA]
A: V | AD: V | M: V | F: V | T: V
(A and AD and M and F) entonces T = V

[SALDOS FINALES]
Juan Perez | Saldo: S/ 750 | Activa: V
Maria Lopez | Saldo: S/ 800 | Activa: V

TABLA DE VERDAD: (A and M and F) entonces R

Caso  A      M      F      R      Formula
1      F      F      F      F      V
2      F      F      V      F      V
3      F      V      F      F      V
4      F      V      V      F      V
5      V      F      F      F      V
6      V      F      V      F      V
7      V      V      F      F      V
8      V      V      V      V      V
```

4. Métricas de cobertura lógica.

[METRICAS]
Casos posibles: 8
Casos evaluados: 8
Casos correctos: 8
Casos fallidos: 0
Retiros realizados: 1
Retiros denegados: 7
Cobertura logica: 100%

Anexo E. Evidencia del repositorio

